

CO-OCCURENCE NETWORK LINK PREDICTION

Ibrahim Al Khalil RIDENE

ibrahim-al-khalil.ridene@student-cs.fr

Ulysse Roux

ulysse.roux@student-cs.fr

Théau d’Audiffret

theau.daudiffret@student-cs.fr

Syphax Yesli

syphax.yesli@student-cs.fr

Kaggle Team Name: théau_daudiffret

Abstract

This project addresses the problem of missing link prediction in an actor co-occurrence network, where nodes represent actors and edges denote co-occurrence on Wikipedia pages. Leveraging both the graph structure and textual features derived from Wikipedia descriptions, we aim to accurately reconstruct the original network. We explore traditional machine learning approaches using manually engineered features and models such as XGBoost and Random Forest. The experimental results demonstrate the effectiveness of combining graph-based deep learning with classic ML techniques for robust link prediction under distribution shift. In parallel, We implement and compare a series of graph neural network (GNN) architectures, including GCN, GAT, GraphSAGE, GIN, APPNP, and Transformer-based models, each combined with handcrafted node-wise statistics and Node2Vec embeddings to enhance predictive performance.

1. Introduction

Link prediction is a key task in network science, aiming to infer missing or future connections between nodes. In this project, we tackle link prediction in an actor co-occurrence graph, where nodes represent actors and edges indicate co-appearance on Wikipedia pages—a proxy for professional or social relationships.

To solve this, we explore both deep learning and traditional machine learning approaches. On the deep learning side, we implement and evaluate several Graph Neural Network (GNN) architectures, including Graph Convolutional Networks (GCN) [5], Graph Attention Networks (GAT) [7], GraphSAGE [4], GIN [8], APPNP [6], and Transformer-based GNNs [9]. These models are further enhanced by integrating Node2Vec embeddings [3] and node-level statistics from graph theory.

In parallel, we experiment with classical machine learn-

ing models such as XGBoost [2] and Random Forests [1] trained on handcrafted topological features.

This dual approach allows us to compare modern GNN-based methods with lightweight ML baselines for robust link prediction in sparse and partially observed networks.

2. Feature Engineering

2.1. Machine Learning Approach

To effectively perform link prediction, we engineered two types of features, topological features inherent to the structure of the graph and node features based on the node embeddings provided.

2.1.1. Topological Features

- **Node degree**
- **Common Neighbors (CN):** Number of shared neighbors
- **Jaccard Coefficient (JC):** $JC(u, v) = \frac{|N(u) \cap N(v)|}{|N(u) \cup N(v)|}$.
- **Adamic-Adar Index (AA):**
 $AA(u, v) = \sum_{w \in N(u) \cap N(v)} \frac{1}{\log |N(w)|}$.
- **Preferential Attachment (PA):**
 $PA(u, v) = |N(u)| \cdot |N(v)|$.
- **Resource Allocation Index (RA):**
 $RA(u, v) = \sum_{w \in N(u) \cap N(v)} \frac{1}{|N(w)|}$.
- **Mean of the two nodes clustering coefficients**
- **Betweenness Centrality:** Measures of centrality of the node.
- **PageRank Score:** It gives a score to each node based on the idea that connections from important nodes contribute more to the score.
- **Resource Allocation Score:** It evaluates similarity based on shared neighbors, weighting rare neighbors more:
 $RA(u, v) = \sum_{w \in N(u) \cap N(v)} \frac{1}{|N(w)|}$.
- **Authority Score**
- **Hub Score**

2.1.2. Node Features

Three approaches were conducted:

- **Similarity features:** We computed the cos similarity, the sum of the embedding, the mean, the co-occurrence of 1 in both embedding.
- **Dimensionality reduction:** We used Truncated SVD to reduce the dimension of the embedding and thus be able to fit them as features of our XGboost.
- **CNN approach:** We used CNN to capture local dependencies between the two embedding.

These three approaches performed quite well (dimensionality reduction gave 0.68) but did not combine well with the other features so we decided to focus on topological features.

2.1.3. Final features

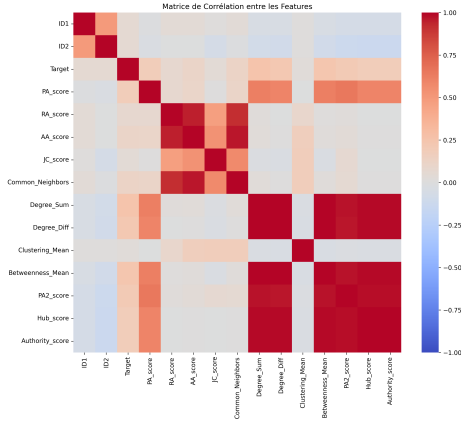


Figure 1. Correlation Matrix between features.

By doing our grid-search of different combinations of features we discovered that the structure of a graph was really close to a hub and spokes model so the most important features is node centrality. Since two central nodes have much more probabilities to be connected, preferential attachment turned out to be the best feature with good generalization capabilities. All other features mentioned before were too correlated with preferential attachment to perform well, the only feature not too correlated was resource allocation which enables to grasp even more positive connections by attributing more importance between two nodes when the degree of the nodes are low. This shed light on connections outside of the obvious hub links.

2.2. Deep Learning Approach

While traditional machine learning models such as Random Forest and XGBoost heavily rely on effective feature design to perform well, deep learning models are generally capable of learning useful representations directly from raw input data. In the context of graph-based models, Graph Neural Networks (GNNs) can learn to combine and filter node features and neighborhood information through multiple message-passing layers, often making manual feature

engineering unnecessary if the input features are sufficiently informative.

However, in our problem setting, the provided node features derived from Wikipedia keyword vectors alone are not always sufficient to enable the network to distinguish structural roles or graph connectivity patterns between nodes. To enhance the learning capacity of our models, we introduced a set of handcrafted features that aim to capture complementary graph-theoretic and positional information.

2.2.1. Node-Wise Graph Statistics

We engineered 10 additional features per edge by extracting structural descriptors from the graph constructed using positive training edges. For a given pair of nodes (u, v) , we compute the following features:

- **Degree (deg):** The number of direct neighbors a node has. High-degree nodes are often hubs or well-connected actors.
- **Clustering Coefficient (clu):** Measures how likely a node's neighbors are to be connected. It captures local cohesiveness.
- **PageRank (pr):** Reflects the importance of a node within the graph, accounting for both direct and indirect connections.
- **Closeness Centrality (cls):** Inverse of the average shortest path length from a node to all other nodes. Captures node accessibility.
- **Betweenness Centrality (bet):** Quantifies the number of shortest paths that pass through a node, highlighting bridge nodes.

Each of the above metrics is computed for both nodes u and v in the pair, resulting in $2 \times 5 = 10$ additional features. These statistics help the model understand each node's role in the graph topology and relative positioning, which may not be captured by local GNN message passing.

2.2.2. Unsupervised Node Embeddings: Node2Vec

To capture more nuanced structural similarity beyond first-order neighborhoods, we also integrated **Node2Vec** embeddings [3], an unsupervised method that learns continuous feature representations for nodes using biased random walks. The key intuition behind Node2Vec is to optimize node embeddings such that nodes sharing similar structural roles or appearing in similar network contexts have embeddings that are close in the vector space.

We trained a Node2Vec model on the union of the training and test graph edges, generating 64-dimensional embeddings for each node. These embeddings were then added to the GNN-based representations during training and evaluation, effectively enriching the learned features with unsupervised structural priors.

2.2.3. Impact of Feature Combinations

To evaluate the effectiveness of our feature engineering, we conducted experiments comparing different combinations of features:

- Using only the original keyword features from `node_information.csv`.
- Adding node-wise statistics only.
- Adding Node2Vec embeddings only.
- Combining all three sources: keyword features, Node2Vec embeddings, and node-wise statistics.

Our results show that the combination of all three feature types consistently improved validation accuracy across all GNN architectures. While the gains were often modest, they demonstrate that even with powerful deep learning models, carefully crafted structural features can provide valuable complementary signals.

2.2.4. Feature Selection

Since deep learning models are capable of implicit feature selection through gradient-based learning, we did not apply explicit feature selection methods.

3. Model Tuning and Comparison

3.1. Machine Learning Approach

At first, we tried to mix topological features and Node Features in one single Random Forest algorithm [?] but this did not work well. So, we tried to utilize the CNN approach described in 2.1.2 to generate a prediction that is going to be used as a feature of a Random Forest model. Then, we fitted the model using also the preferential attachment feature. Since we did not manage to surpass 0.70 with this method, we concentrated our efforts on topological features. We started by implementing a Random Forest algorithm with 100 voting trees and looked at accuracy and recall as well as the ROC-AUC curve to measure the balance of our model when predicting true positive and false positive. Then, we decided to use XGBoost [2] to benefit from boosting and thus learn more precise patterns especially to be able to use the resource allocation feature as well. At this stage, the model started to perform well (0.75 approximately). Then we increased the number of estimators to gain the last percentages. Trees of depth 6 turned out to offer the best bias-variance trade-off. We used logloss to train the model not only on the result of the classification but also on the probabilities associated with the class. It helps building a coherent model.

3.2. Deep Learning Approach

3.2.1. Data Splitting Strategy

To train and evaluate our models fairly, we split the labeled pairs from `train.txt` into a training and validation set with a 90/10 ratio. To maintain class balance, the split was

stratified separately for positive and negative examples. The test pairs from `test.txt` were used exclusively for final evaluation. All node IDs were mapped to continuous indices, and node features were extracted from the keyword-based `node_information.csv` file.

3.2.2. Model Comparison

We evaluated six GNN-based models: GCN, GAT, GraphSAGE, GIN, APPNP, and TransformerGNN. Each model was trained with the same optimization pipeline and feature setup, including given features, node-wise statistics and Node2Vec embeddings.

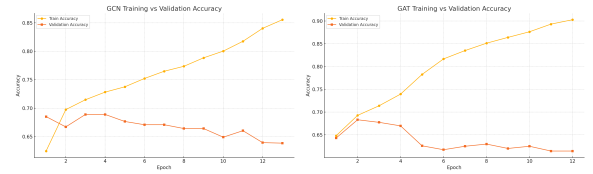


Figure 2. GCN vs. GAT Training and Validation Accuracy

Analysis: Both GCN and GAT exhibit strong training accuracy but suffer from overfitting, with validation accuracy stagnating or slightly degrading after early epochs. GAT shows the steepest train curve, but its generalization degrades earlier.

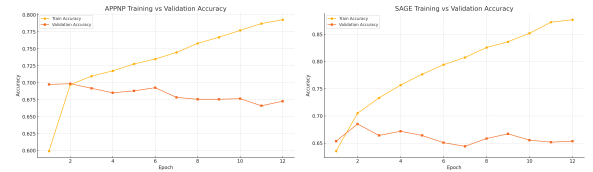


Figure 3. APPNP vs. GraphSAGE Training and Validation Accuracy

Analysis: APPNP maintains the best validation accuracy curve overall and displays more stable generalization. GraphSAGE also performs well, though validation accuracy oscillates mid-training.

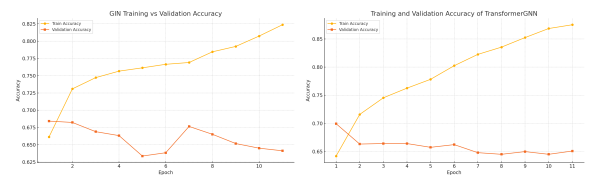


Figure 4. GIN vs. TransformerGNN Training and Validation Accuracy

Analysis: GIN converges smoothly but underperforms on validation. TransformerGNN suffers from overfitting.

Model	Validation Accuracy	Test Accuracy
TransformerGNN	0.699	0.7487
APPNP	0.6985	0.7560
GCN	0.689	0.7600
SAGE	0.685	0.7484
GIN	0.684	0.7460
GAT	0.683	0.7620

Table 1. Validation and Test Accuracy Across GNN Models

3.2.3. Validation and Test Performance Summary

Conclusion: GAT and GCN achieved the best test accuracy, but with clear overfitting. APPNP and TransformerGNN offered the best trade-off between validation and test performance, suggesting stronger generalization.

3.2.4. Tuning and Overfitting Prevention

To optimize model performance and prevent overfitting, the following strategies were applied:

- **L2 Regularization:** Applied via weight decay in the optimizer to penalize large weights.
- **Early Stopping:** Training halted if validation accuracy did not improve for a fixed number of epochs (patience).
- **Best Model Saving:** The model with the highest validation accuracy was saved and used for final testing.
- **Dropout:** Applied on GNN layers and adjacency matrix (via edge dropout) to prevent over-reliance on specific graph connections.

These regularization techniques contributed to maintaining generalization across all architectures and allowed robust performance comparisons.

4. Conclusion

Studying the problem of link prediction in the context of actor co-occurrence in Wikipedia pages we found out that some actors were highly connected while others were only connected to few hubs. This distribution is quite logical since some stars like Brad Pitt will be connected to many actors he played with while the unknown actors that played with him will only be connected to him. That’s why exploiting the topology of the graph as well as using Graph Neural Networks worked so well because these techniques grasped the essence of the graph. The embedding of the nodes surely contained some meaningful information but the correlation between the features computed from the embedding and the topological features prevented a good generalization. As, a result, our best model performing with 0.76851 in Kaggle, is the Machine Learning model revealing that the structure of the graph was maybe quite simple and enabled some very basic features to perform well. We are currently mixing the two approaches by using the last layer of a GNN to compute features that are added to our two best performing features.

Then we plan to feed this to a meta model which will surely be an XGBoost given its performances and thus we hope to beat our current best.

References

- [1] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. [1](#)
- [2] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 785–794, 2016. [1](#), [3](#)
- [3] Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 855–864. ACM, 2016. [1](#), [2](#)
- [4] William L Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 1025–1035, 2017. [1](#)
- [5] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017. [1](#)
- [6] Johannes Klicpera, Aleksandar Bojchevski, and Stephan Günnemann. Predict then propagate: Graph neural networks meet personalized pagerank. In *International Conference on Learning Representations (ICLR)*, 2019. [1](#)
- [7] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018. [1](#)
- [8] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019. [1](#)
- [9] Seunghyun Yun, Minbyul Jeong, Raehyun Kim, Jaewoo Kang, and Hyunwoo J Kim. Graph transformer networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. [1](#)